

Final Project IEMS351

Group 5

December 2024

Summary

In the final project, we thoroughly reviewed the Stochastic Cubic Regularized Newton Method. First, we analyzed the strengths and weaknesses of the Stochastic Cubic Regularization (SCR) algorithm, and examined the algorithm's efficiency in escaping saddle points, its improved complexity and its simplicity in implementation without requiring variance reduction techniques. However, we also noted its reliance on stochastic gradients and Hessians, potential noise issues, and the need for precise tuning of parameters.

To evaluate the algorithm's practical performance, we introduced the Rosenbrock function, a non-convex function fulfilling the assumptions and limitations given by the paper. Specifically, this function satisfies the smoothness assumptions outlined in the paper, including ℓ -Lipschitz gradients and ρ -Lipschitz Hessians. We constructed synthetic data and applied SCR while comparing it to benchmark algorithms such as Stochastic Gradient Descent (SGD) and AdaGrad.

We implemented SCR by adhering to the algorithm described in the paper, using stochastic gradients and Hessian-vector products, and evaluated its performance under varying hyperparameter settings. Our results demonstrated that SCR outperformed SGD and AdaGrad in terms of convergence speed on the Rosenbrock function.

Last but not least, we conducted a sensitivity analysis to see how key hyperparameters such as regularization strength (ρ), step size (η), and momentum (β), influenced the convergence speed across our algorithms. This analysis that we did provided insights into the robustness of paper's SCR and its sensitivity to different parameter settings. Through benchmarking, we validated that SCR achieves better performance in non-convex optimization problems.

Contributions

Each team member contributed to the project as such:

- **Mulan:** Implemented the SCR algorithm, compared its performance with benchmark algorithms such as SGD and AdaGrad, analyzed the results,

and conducted sensitivity analysis. Mulan also played a key role in ensuring the experiments aligned with the assumptions of the paper and helped interpret the outcomes in relation to the original methodology.

- **Zihan:** Described the problem and data sources, provided the motivation for choosing this model, explained its construction using ϵ -approximate local minima and cubic regularization, discussed the algorithm’s complexity, and detailed the parameters used in the generated data. Zihan also implemented the benchmark algorithms such as AdaGrad, and worked on summarizing the theoretical underpinnings and connecting them to the experimental results.
- **Alan:** Built and implemented the benchmark algorithms such as SGD, programmed the code for these algorithms, and ensured proper integration with the SCR implementation. Alan also contributed by writing the detailed summary of key aspects of the paper, as well as assisting with the debugging efforts of the implementations throughout the project.

1 Paper Review

1.1 Strengths and Weakness

After a thorough review of the paper, we identified that the major strengths of the stochastic cubic-regularized Newton Method mostly lies in several areas. First, the method enables an extremely efficient escape from saddle points, which is not usually doable if using traditional methods such as SGD. This is because the stochastic cubic-regularized Newton Method leverages curvature information through stochastic Hessian-vector product, which is significantly more efficient. Another major source of strength is the algorithm’s improved complexity: $O(\epsilon^{-3.5})$. This outperforms SGD’s $O(\epsilon^{-4})$ rate, and therefore converges faster to local minima in various settings, enabling it to perform better in high-dimensional settings. Last but not least, we identified that the stochastic cubic-regularized Newton method is easy to implement in that it computes stochastic Hessian-vector products as efficiently as stochastic gradient, which is scalable for large scale problems, and also unlike other methods such as Natasha 2, the algorithm does not require complex variance reduction technique, which simplifies implementation.

Despite the advantages of this algorithm, we also targeted several weaknesses of the algorithm. First, it relies on stochastic gradient and Hessians, which could introduce noise and affect the performance when there is high variance. Second, we identified that solving the cubic submodel inexactly could also result in subpar parameter updates, and that depends on the tolerance setting for the subsolver, which could also introduce sources of errors. The algorithm also requires careful tuning of parameters such as batch sizes and tolerance for the subsolver, and that could be time-consuming and not so easily usable.

1.2 Reproducibility

While we did not use the exact experiments conducted in the original paper due to computational constraints (after consulting Prof. Diao), we designed and implemented an experiment that satisfies the exact same assumptions and requirements mentioned in the paper.

Specifically, we used the Rosenbrock function, a classic non-convex function as a test case that follows the smoothness conditions described in the paper, including ℓ -Lipschitz gradients and ρ -Lipschitz Hessians. This alternative experimental setup allowed us to evaluate the Stochastic Cubic Regularization (SCR) algorithm under comparable theoretical conditions, ensuring the reproducibility of the core results. By focusing on an experiment that fulfilled the necessary assumptions, we maintained the integrity of the algorithm's evaluation while adapting to computational limitations. This approach provided meaningful insights into the performance and applicability of SCR without compromising the validity of our findings.

2 Numerical Experiment

2.1 Problem Description

We aim to minimize a smooth, non-convex function $f(x)$:

$$f(x) := E_{\xi \sim D} f(x; \xi)$$

We use the stochastic gradients $\nabla f(x; \xi)$ and stochastic Hessian matrices $\nabla^2 H(x; \xi)$ to achieve this approximation framework.

Previously, most people chose to use the stochastic gradient decent method (SGD) to solve non-convex problems, but it is computationally inefficient. We design an algorithm called stochastic cubic regularized methods to find the ϵ -approximate local minimum.

First, we define the concept of ϵ -approximate local minimum: for a ρ -Hessian Lipschitz function f , x is an ϵ -approximate local minimum if

$$\|\nabla f(x)\| \leq \epsilon \quad \text{and} \quad \lambda_{\min} \geq -\sqrt{\rho\epsilon}$$

From the equation, we can find an ϵ -approximate local minimum not only has a small gradient, but also has a Hessian matrix that is close to positive semidefinite, as ϵ is small and cubic regularized term ρ is a constant.

According to the **Second order necessary optimality condition**, for a twice continuously differentiable function $f: R^n \rightarrow R$, if x^* is a local minimum, then $\nabla f(x^*) = 0$ and $\nabla^2 f(x^*)$ is positive semidefinite. So, the ϵ -approximate local minimum is the approximate local minimum for our function.

To show the effectiveness of our algorithms, we did comparison and used our own function. We chose the Rosenbrock function as our optimization function. The function is

$$f(x) = (1 - x)^2 + (3y - x^2)^2$$

.And then we compute the gradient $\nabla f(x, y)$:

$$\begin{aligned}\frac{\delta f}{\delta x} &= -2(1-x) - 6x(y-x^2) \\ \frac{\delta f}{\delta y} &= 6(y-x^2)\end{aligned}$$

. We compute the result when $\nabla f(x, y) = 0$ and get the $x, y = (1, 1)$. Then we plug these values into the Hessian matrix and compute eigenvalues.

$$H = \begin{bmatrix} 2 + 6(y-x^2) - 12x^2 & -12x \\ -12x & 6 \end{bmatrix} = \begin{bmatrix} -10 & -12 \\ -12 & 6 \end{bmatrix}$$

Convexity: According to the **Second-order sufficient condition:** for a twice continuously differentiable function $f: R^n \rightarrow R$, if $\nabla f(x^*) = 0$ and $\nabla^2 f(x^*)$ is indefinite, then x^* is saddle point. We compute $\det(H)$ for our function and get two eigenvalues $r_1 = 24.42$, $r_2 = -4.42$. Since $r_2 \leq 0$, the Hessian matrix is indefinite, which implies that the critical point at $(1, 1)$ is a saddle point. Since our problem is non-convex problem, the function doesn't follow the optimality conditions here.

Smooth: From the perspective of continuity, the Rosenbrock function is composed of polynomial terms (quadratic and quartic), which are continuous functions. Therefore, the Rosenbrock function is continuous over the entire domain of x and y . From the perspective of derivatives, the function has continuous derivatives of all orders, without any sharp corners or discontinuities. As a result, the Rosenbrock function is continuous and has continuously derivatives of all orders, which means this function is smooth.

Theorem: There exists an absolute constant c such that if $f(x)$ satisfies Assumptions 1, 2, CubicSubsolver satisfies Condition 1 with

$$n_1 \geq \max \left(\frac{M_1}{c\epsilon}, \frac{\sigma_1^2}{c^2\epsilon^2} \right) \log \left(\frac{d\sqrt{\rho}\Delta_f}{\epsilon^{1.5}\delta c} \right),$$

and

$$n_2 \geq \max \left(\frac{M_2}{c\sqrt{\rho}\epsilon}, \frac{\sigma_2^2}{c^2\rho\epsilon} \right) \log \left(\frac{d\sqrt{\rho}\Delta_f}{\epsilon^{1.5}\delta c} \right),$$

then for all $\delta > 0$ and $\Delta_f \geq f(x_0) - f^*$, Algorithm 1 will output an ϵ -second-order stationary point of f with probability at least $1 - \delta$ within

$$\tilde{O} \left(\frac{\sqrt{\rho}\Delta_f}{\epsilon^{1.5}} \left(\max \left(\frac{M_1}{\epsilon}, \frac{\sigma_1^2}{\epsilon^2} \right) + \max \left(\frac{M_2}{\sqrt{\rho}\epsilon}, \frac{\sigma_2^2}{\rho\epsilon} \right) \cdot \mathcal{T}(\epsilon) \right) \right)$$

total stochastic gradient and Hessian-vector product evaluations.

Complexity: From the theory above and computation, if $\epsilon \leq \min \left\{ \frac{\sigma_1^2}{c_1 M_1}, \frac{\sigma_2^4}{c_2^2 M_2^2 \rho} \right\}$, we can conclude that our algorithm will output an ϵ -second-order approximate local minimum of $f(\mathbf{x})$ within

$$\tilde{O} \left(\frac{\sqrt{\rho}\Delta_f}{\epsilon^{1.5}} \left(\frac{\sigma_1^2}{\epsilon^2} + \frac{\sigma_2^2}{\rho\epsilon} \cdot \frac{\ell}{\sqrt{\rho}\epsilon} \right) \right)$$

To analyze the complexity, let's simplify the expression inside the big O notation. The term $\frac{\sigma_1^2}{\epsilon^2}$ is already in the simplest form. For the second term, we can simplify $\frac{\sigma_2^2}{\rho\epsilon} \cdot \frac{\ell}{\sqrt{\rho\epsilon}}$ as $\frac{\sigma_2^2\ell}{\rho^{3/2}\epsilon^{3/2}}$. We observe that the dominant term of the submodel is $\frac{\sigma_1^2}{\epsilon^2}$ from the first term. Therefore, the overall complexity is $\tilde{O}(\epsilon^{-3.5})$.

2.2 Data Source

To simplify the problem, the data set is generated by our own. The dataset only contains the initial point at first. We use 100 initial samples at first. As the number of iterations increases, we continue to generate new x and y, and these x and y are closer to the optimal point (loss decreased).

We used the function containing two different variables x and y. So the dimension of the function is 2. We define that the ranges of x and y are both $\mathcal{R}$. And all parameters in our model are shown below:

$x_{\text{init}}, y_{\text{init}}$	[0.5, 0.5]
ϵ	10^{-2}
ρ	10.0
l	1.0
η	0.01
Iterations	20
n_1, n_2	5, 5
Dataset Size	100 samples

Table 1: Training parameters Table

$x_{\text{init}}, y_{\text{init}}$: the initial estimated solution of x and y
 ϵ : convergence threshold
 ρ : the cubic-regularized term
 η : the step size of the parameter updates during each iteration.
Iterations: total iteration times.
 n_1, n_2 : mini-batch size of our algorithms
Dataset Size: the number of samples in our experiment.

2.3 Algorithm Implementation

The algorithm we implemented is the Stochastic Cubic Regularization (SCR) method, designed to solve optimization problems for smooth, non-convex functions efficiently. Below is the pseudocode of the SCR algorithm, adapted from the original paper:

Algorithm 1 Stochastic Cubic Regularization (Meta-algorithm)

Input: Mini-batch sizes n_1, n_2 , initialization x_0 , number of iterations T_{out} , and final tolerance ϵ .

Output: x^* if the early termination condition was reached; otherwise, the final iterate $x_{T_{\text{out}}+1}$.

```
1: for  $t = 0, \dots, T_{\text{out}}$  do
2:   Sample  $S_1 \leftarrow \{\xi_i\}_{i=1}^{n_1}, S_2 \leftarrow \{\xi_i\}_{i=1}^{n_2}$ 
3:    $g_t \leftarrow \frac{1}{|S_1|} \sum_{\xi_i \in S_1} \nabla f(x_t; \xi_i)$ 
4:    $B_t[\cdot] \leftarrow \frac{1}{|S_2|} \sum_{\xi_i \in S_2} \nabla^2 f(x_t; \xi_i)(\cdot)$ 
5:    $\Delta, \Delta_m \leftarrow \text{Cubic-Subsolver}(g_t, B_t[\cdot], \epsilon)$ 
6:    $x_{t+1} \leftarrow x_t + \Delta$ 
7:   if  $\Delta_m \geq -\frac{1}{100} \sqrt{\frac{\epsilon^3}{\rho}}$  then
8:      $\Delta \leftarrow \text{Cubic-Finalsolver}(g_t, B_t[\cdot], \epsilon)$ 
9:      $x^* \leftarrow x_t + \Delta$ 
10:    break
11:  end if
12: end for
```

The algorithm consists of the following key steps. First, mini-batches are sampled to compute the stochastic gradient, g_t , and Hessian-vector products, $B_t[\cdot]$, required for updating the parameters. Next, the cubic submodel is defined as:

$$m_t(\Delta) = \Delta^\top g + \frac{1}{2} \Delta^\top B[\Delta] + \frac{\rho}{6} \|\Delta\|^3,$$

where ρ is the regularization parameter. The cubic submodel is then solved approximately with a certain degree of suboptimality to determine the step Δ and the corresponding submodel change Δ_m . After computing Δ , the algorithm updates the parameters using $x_{t+1} = x_t + \Delta$. To maintain consistency with the original paper, we specifically used gradient descent as the optimization approach for the subsolver, ensuring it meets the necessary conditions stated in the paper. The pseudocode for the subsolver and final solver is shown below:

Algorithm 2 Cubic-Subsolver via Gradient Descent

Input: $g, B[\cdot]$, tolerance ϵ .**Output:** Δ, Δ_m .

```
1: if  $\|g\| \geq \frac{\epsilon^2}{\rho}$  then
2:    $R_c \leftarrow -\frac{g^\top B[g]}{\rho\|g\|^2} + \sqrt{\left(\frac{g^\top B[g]}{\rho\|g\|^2}\right)^2 + \frac{2\|g\|}{\rho}}$ 
3:    $\Delta \leftarrow -R_c \frac{g}{\|g\|}$ 
4: else
5:    $\Delta \leftarrow 0, \sigma \leftarrow c' \sqrt{\frac{\epsilon\rho}{\ell}}, \eta \leftarrow \frac{1}{20\ell}$ 
6:    $\tilde{g} \leftarrow g + \zeta$  for  $\zeta \sim \text{Unif}(S^d - 1)$ 
7:   for  $t = 1, \dots, T(\epsilon)$  do
8:      $\Delta \leftarrow \Delta - \eta(\tilde{g} + B[\Delta] + \frac{\rho}{2}\|\Delta\|\Delta)$ 
9:   end for
10: end if
11:  $\Delta_m \leftarrow g^\top \Delta + \frac{1}{2}\Delta^\top B[\Delta] + \frac{\rho}{6}\|\Delta\|^3$ 
```

Algorithm 3 Cubic-Finalsolver via Gradient Descent

Input: $g, B[\cdot]$, tolerance ϵ .**Output:** Δ .

```
1:  $\Delta \leftarrow 0, g_m \leftarrow g, \eta \leftarrow \frac{1}{20\ell}$ 
2: while  $\|g_m\| > \frac{\epsilon}{2}$  do
3:    $\Delta \leftarrow \Delta - \eta g_m$ 
4:    $g_m \leftarrow g + B[\Delta] + \frac{\rho}{2}\|\Delta\|\Delta$ 
5: end while
```

Finally, the algorithm checks the termination condition. If Δ_m satisfies the condition, the solution x^* is returned; otherwise, the process continues until convergence or the maximum number of iterations is reached.

2.4 Benchmark Algorithms

We compared the Stochastic Cubic Regularization (SCR) algorithm against two benchmark algorithms: **AdaGrad** and **Stochastic Gradient Descent (SGD)**. These two algorithms were also used in the paper as benchmarks. Our goal is to verify our results and determine if they are consistent with the findings in the paper.

One of our benchmark algorithms, Adagrad, is an adaptive learning rate method and a predecessor of modern optimizers Adam. By scaling learning rates inversely proportional to the square root of historical gradient magnitudes, AdaGrad is particularly effective for sparse data but suffers from diminishing learning rates over time.

Another benchmark algorithm we used is SGD, a simple yet widely used optimization method. It iteratively updates model parameters using gradients

computed from random data samples. However, it often struggles with convergence in high-dimensional non-convex problems.

Figure 1 illustrates the convergence behavior of the three algorithms over 20 iterations. The MSE loss function values are plotted against the number of iterations on the Rosenbrock function. SCR demonstrates significantly faster convergence and achieves the lowest final loss compared to both AdaGrad and SGD.

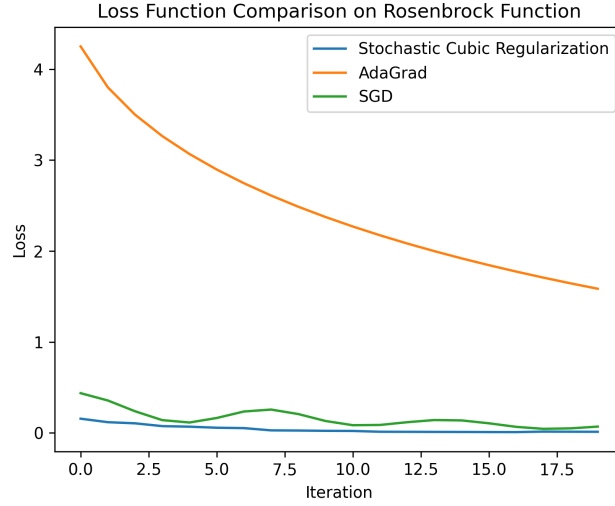


Figure 1: Loss Function Comparison on Rosenbrock Function.

SCR’s final loss of 0.0118 is significantly lower than AdaGrad’s 1.5854 and SGD’s 0.0698. AdaGrad suffers from a high initial loss, which limits its overall performance. On the other hand, SGD converges relatively faster but struggles to effectively escape saddle points and exhibits oscillations in the loss, likely due to its stochastic nature, making the direction of updates vary across iterations.

In summary, SCR outperforms the benchmark algorithms and demonstrates favorable performance on synthetic data.

2.5 Results

2.5.1 Numerical results

The findings largely align with our expectations and corroborate the results presented in the original paper. As highlighted in the original study, SCR effectively escapes saddle points and converges to the global minimum faster than SGD on synthetic problems. Moreover, our experiments demonstrated that SCR outperforms AdaGrad in both convergence speed and final loss, a comparison not addressed in the paper.

To validate the accuracy of our solutions, we recorded the initial loss, final loss, and average loss for each algorithm across multiple runs to assess their performance. These metrics were calculated based on the loss function values at the start, end, and throughout the optimization process. Table 2 summarizes the results:

Algorithm	Initial Loss	Final Loss	Average Loss
SCR	0.1569	0.0118	0.0334
AdaGrad	4.2500	1.5854	2.6347
SGD	0.4375	0.0698	0.1608

Table 2: Performance comparison of SCR, AdaGrad, and SGD over 20 iterations.

2.5.2 Sensitivity Analysis

Due to computational constraints, performing a full grid search across all hyperparameters was infeasible. To address this, we fixed the regularization parameter ($l = 1.0$) and the maximum number of iterations ($\text{max_iterations} = 50$). This allowed us to focus on analyzing the relationship between convergence speed (iterations to converge) and key hyperparameters across the three optimization algorithms. Specifically, we conducted a grid search over the following hyperparameter values:

- ϵ (Convergence threshold): $\{10^{-2}, 10^{-3}\}$
- ρ (Regularization strength, SCR): $\{1.0, 10.0\}$
- η (Step size, AdaGrad and SGD): $\{0.01, 0.1\}$
- β (Momentum, SGD): $\{0.0, 0.9\}$

Table 3 highlights the key hyperparameters influencing each algorithm and those to which they are most sensitive.

Algorithm	Key Hyperparameters	Most Sensitive To
SCR	ρ, ϵ	ρ
AdaGrad	ϵ, η	η
SGD	ϵ, η, β	η, β

Table 3: Key hyperparameters and sensitivity for each algorithm.

Figure 2 shows the correlation between hyperparameters and the convergence speed (number of iterations required for convergence). While the correlation may not be entirely precise due to the imposed limit on the maximum number of iterations, the direction and relative magnitude of the correlations

still provide insights into identifying the most influential parameters for each algorithm. These insights reinforce the importance of hyperparameters tuning to optimize the performance of each algorithm.

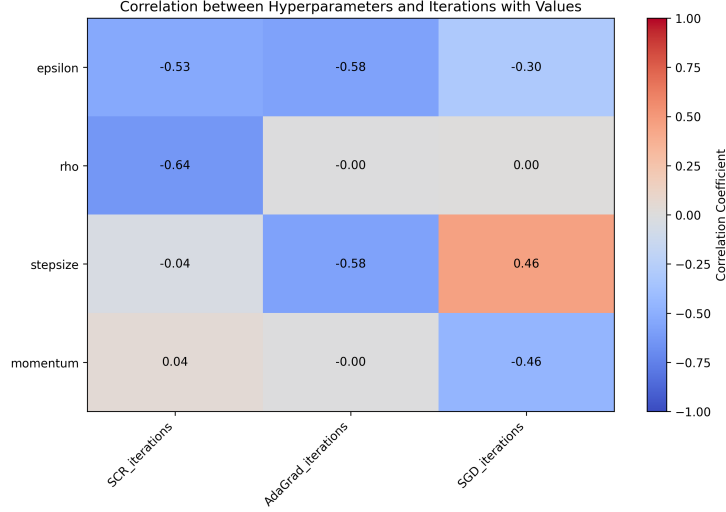


Figure 2: Correlation between hyperparameters and iterations required for convergence.

The sensitivity analysis highlights the significant role of hyperparameter selection in determining the convergence efficiency of optimization algorithms. While SCR demonstrates robustness, careful tuning of ρ is crucial. Similarly, AdaGrad and SGD rely heavily on step size and momentum parameters to achieve optimal performance.

3 Future Directions

3.1 Improvements and New Applications

We believe there are several potential ways to expand the work on this paper for SCR algorithm. An important next step is to test the algorithm on more functions and real-world applications that are more complex and sophisticated, in order to really observe the practicality of this approach, ensuring that we could see the whole picture of the algorithm’s robustness. Meanwhile, we could also compare SCR with other state-of-the-art optimization methods to gain a clearer picture of its strengths and weaknesses.

From another perspective, we could adapt the algorithm to more structured non-convex problems such as the ones mentioned in the reinforcement learning and generative modeling guest lecture, where there are frequently challenges where current assumptions of the SCR may not hold, which requires more

changes to make the algorithms suitable to these situations.